

数据结构第三次作业

3.6

反设存在这样的输出，即最先输出的是最大的数 P_i ，那么后面两个数 P_j 、 P_k 还在栈中，由压栈操作及 $P_k > P_j$ ， P_j 一定被先压入栈中，再由出栈顺序，无论如何， P_j 一定后出栈，与 $j < k$ 矛盾。反设不成立，得证。

3.21

```
//我是把每一个字符当作一个数字接收
//输出方面，其实这里更好的方式应该是把逆波兰序列存进一个数组返回，我是直接在函数里面输出了
#define Elemtyp e char
int Is_OP(char c){
    return c == '+' || c == '-' || c == '*' || c == '/' || c == '(' || c == ')' || c == '#';
}

void Trans(char *str){
    int KVtable[128] = {0};
    KVtable['+'] = 0;
    KVtable['-'] = 1;
    KVtable['*'] = 2;
    KVtable['/'] = 3;
    KVtable['('] = 4;
    KVtable[')'] = 5;
    KVtable['#'] = 6;

    //用1表示'>',0表示'=',-1表示'<',2表示不会有这个情况
    int relation[7][7] = {
        {1,1,-1,-1,-1,1,1},
        {1,1,-1,-1,-1,1,1},
        {1,1,1,1,-1,1,1},
        {1,1,1,1,-1,1,1},
        {-1,-1,-1,-1,-1,0,2},
        {1,1,1,1,2,1,1},
        {-1,-1,-1,-1,-1,2,0}
    };

    SqStack OPTR;
    InitStack(&OPTR);

    Push(&OPTR, '#');
    str++;
    char c = *str;
    while(GetTop(OPTR) != '#' || *str != '#'){
        c = *str;
        //是操作符
        if(Is_OP(c)){
            char top_op = GetTop(OPTR);
            int i = KVtable[top_op], j = KVtable[c];
```

```

        if(relation[i][j] == 1){
            char x;
            Pop(&OPTR,&x);
            printf("%c ",x);
        }
        else if(relation[i][j] == 0){
            char x;
            Pop(&OPTR,&x);
            str++;
        }
        else{
            Push(&OPTR,c);
            str++;
        }
    }
    //是数字
    else{
        printf("%c ",c);
        str++;
    }
}

printf("\n");
}

```

3.22

```

//数组定义与上一题一致，不再重复
int Calculate(int x,int y,char OP){
    switch (KVtable[OP])
    {
        case 0:
            return x + y;
        case 1:
            return x - y;
        case 2:
            return x * y;
        case 3:
            return x / y;
        default:
            return ERROR;
    }
}

//处理形如AB+CD*E/-的字符串后缀表达式，ABC等视作操作数
int CalPostfix(char *str){
    char c;
    SqStack OPND;
    InitStack(&OPND);

    while(*str){

```

```
        c = *str++;

        if(Is_OP(c)){
            int x,y;
            Pop(&OPND,&y);
            Pop(&OPND,&x);
            Push(&OPND,Calculate(x,y,c));
        }
        else
            Push(&OPND,c);
    }

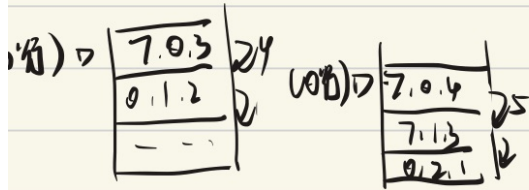
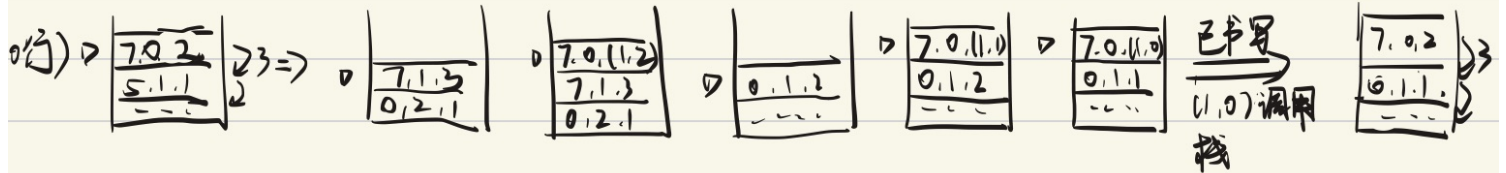
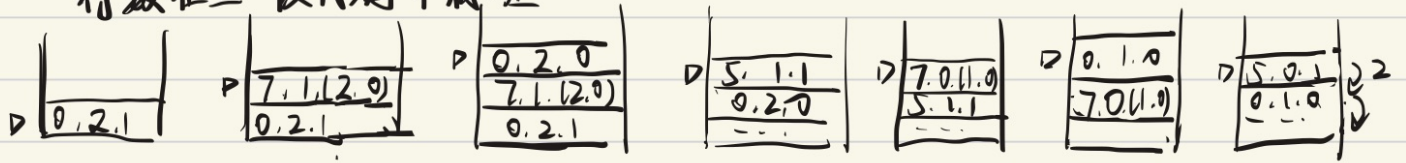
    return GetTop(OPND);
}
```

3.27

3.27(1)

```
int akm(int m,int n){
    if(!m) //1
        return n + 1; //2
    //3
    if(!n) //4
        return akm(m-1,1); //5
    //6
    return akm(m-1,akm(m,n-1)); //7
}
```

三个分别表示回到第几行, m, n . (a, b) 表示 $akm(a, b)$ 的值
 行数在上一段代码中标迹



故 $akm(2, 1) = 5$